

计算机网络小组作业

Illusionna 2025.....04 m4rsquake 2025.....1

02:12, Saturday 6th December, 2025

目录

1	Linux、Windows、macOS 跨平台 C 语言头文件库设计	1
2	简易 HTTP 1.1 Web 服务器	2
2.1	项目描述	2
2.2	实验拓扑	2
2.3	实验流程	2
2.4	项目	2
2.4.1	代码设计	2
2.4.2	编译执行	4
2.4.3	网络测试	5
2.4.4	效果截图	6
3	基于 UDP 的可靠数据传输	9
3.1	项目描述	9
3.2	拓扑结构	9
3.3	实验流程	9
3.4	项目	9
3.4.1	代码设计	9
3.4.2	编译执行	12
3.4.3	网络测试	13
3.4.4	效果截图	13
4	实验总结	16

1 Linux、Windows、macOS 跨平台 C 语言头文件库设计

HTTP Web 服务器和 UDP 可靠传输协议均采用 C 语言 socket API 实现，而不同主流操作系统设计借口均不一样，除此之外，两个项目还用到了多线程、日志、哈希表等通用接口，为此需要编写一个跨平台的 C 语言头文件库。两个项目代码均开源在 GitHub ¹ 仓库。

针对 socket 库，设计函数：

```
int socket_init();
Socket socket_create(int domain, int type, int protocol);
void socket_close(Socket s);
void socket_destroy();
void socket_config(struct sockaddr_in *server, int domain, char *ip, int port);
int socket_connect(Socket s, struct sockaddr_in *server, int size);
int socket_send(Socket s, char *buffer, int length, int flag);
int socket_sendto(Socket s, void *buffer, int length, int flag, struct sockaddr_in *to, int size);
int socket_recv(Socket s, char *buffer, int length, int flag);
int socket_recvfrom(Socket s, void *buffer, int length, int flag, struct sockaddr_in *from, int *size);
int socket_bind(Socket s, struct sockaddr_in *address_name, int size);
int socket_listen(Socket s, int backlog);
Socket socket_accept(Socket s, struct sockaddr_in *address, int *size_pointer);
int socket_setopt(Socket s, int level, int optname, void *ctx, int size);
unsigned int socket_ntohl(unsigned int value);
unsigned int socket_htonl(unsigned int value);
unsigned short socket_ntohs(unsigned short value);
unsigned short socket_htons(unsigned short value);
```

针对 thread 多线程库，设计函数：

```
int thread_create(Thread *thread, _ThreadFunction func, void *args);
int thread_join(Thread *thread, int *result);
int thread_detach(Thread *thread);
void thread_exit();
int mutex_create(Mutex *mutex, int type);
void mutex_destroy(Mutex *mutex);
int mutex_lock(Mutex *mutex);
int mutex_unlock(Mutex *mutex);
```

针对 log 日志库，设计函数：

```
#define log_trace(...) __log_print__(LOG_TRACE, __FILE__, __LINE__, __VA_ARGS__)
#define log_debug(...) __log_print__(LOG_DEBUG, __FILE__, __LINE__, __VA_ARGS__)
#define log_info(...) __log_print__(LOG_INFO, __FILE__, __LINE__, __VA_ARGS__)
#define log_warning(...) __log_print__(LOG_WARNING, __FILE__, __LINE__, __VA_ARGS__)
#define log_error(...) __log_print__(LOG_ERROR, __FILE__, __LINE__, __VA_ARGS__)
#define log_fatal(...) __log_print__(LOG_FATAL, __FILE__, __LINE__, __VA_ARGS__)

void log_config_write(FILE *f);
void log_setting(int mode);
void log_add_callback(_LogCallbackFunc func, void *ctx);
void log_config_thread_lock(_LogLockFunc func, void *ctx);
void __log_print__(int level, char *file, int line, char *fmt, ...);
void __log_init_event__(LogEvent *event, void *ctx);
void __log_lock__();
void __log_unlock__();
void __log_callback_stdout__(LogEvent *event);
void __log_callback_write__(LogEvent *event);
```

针对 HashMap 字典库，设计函数：

```
HashMap *hashmap_create();
void hashmap_put(HashMap *dict, char *key, char *value);
char *hashmap_get(HashMap *dict, char *key);
int hashmap_remove(HashMap *dict, char *key);
void hashmap_destroy(HashMap *dict);
void hashmap_view(HashMap *dict);
unsigned int __hash_function_ELF__(char *str, int size);
void __hashmap_expand__(HashMap *dict);
```

¹<https://github.com/Illusionna/UCAS-Homework/tree/main/计算机网络>

2 简易 HTTP 1.1 Web 服务器

2.1 项目描述

基于 TCP Sockets，从零开始实现一个简单的 HTTP/1.1 Web 服务器，监听 8080 端口，能够解析 HTTP 请求并响应，对于接收的请求：

- 如果当前目录存在所请求的文件，返回状态码 200，以及所请求的文件；
- 如果所请求的为部分内容（请求有 Range 字段），返回状态码 206，以及相应的部分内容；
- 如果当前目录没有所请求的文件，返回状态码 404。

2.2 实验拓扑

```

                                10.0.0.2
h1 ----- s1 ----- h2
```

2.3 实验流程

- 在 h2 服务器的 xterm 中准备一个 index.html 文件，运行所实现的 HTTP 服务器程序，监听端口号 8080；
- 在 h1 客户端的 xterm 中，使用 curl 向 h2 发起请求；

```

- curl http://10.0.0.2:8080/index.html
- curl -r start-end http://10.0.0.2:8080/index.html -o part.html
- curl http://10.0.0.2:8080/nonexistent.html
```

- 检查 h1 是否成功获取了相应的内容（完整文件、部分文件、404 代码）。

2.4 项目

2.4.1 代码设计

本实验要求基于 TCP socket 从零开始实现一个简单的 HTTP/1.1 Web 服务器。实验拓扑设定为服务器（s1/h2）监听 8080 端口，客户端（h1）通过 IP 10.0.0.2 发起请求，目的是掌握 HTTP 协议的请求解析与响应构建流程。Web 服务器主要实现以下核心功能：

- 静态文件服务：能够解析 GET 请求，若请求的文件（如 index.html）存在，则返回状态码 200 及文件内容；
- 范围请求支持：支持 HTTP Range 头部，当收到如 “Range: bytes=7-12” 的请求时，服务器能够解析起始和结束位置，仅读取并返回文件的指定部分，返回 206 状态码；
- 错误处理：当请求的文件不存在时，服务器能够正确识别并返回 404 Not Found 状态码及相应的错误页面。

在代码实现上，采用了多线程机制处理每一个客户端发来的请求，并引入了哈希表（即键值对字典）缓存机制以提升频繁访问的响应速度，通过 GET 请求路径权限验证，防止客户端访问服务器父级目录文件，制止可能存在的安全隐患。

HTTP Web 服务器代码设计主要过程如下：

Step 1 初始化全局参数

```
#define PORT 8080
long N = 0;
static Mutex mutex;
static volatile int RUNNING = 1;
static Socket S = SOCKET_INVALID;
static HashMap *dict = NULL;
```

Step 2 增加中断信号量

```
// 释放 HashMap 字典，释放 mutex 互斥锁，释放服务器 socket 文件描述符，销毁 socket，重置 while 循环标志符 RUNNING。
void interrupt_handle(int semaphore) {
    hashmap_destroy(dict);
    mutex_destroy(&mutex);
    RUNNING = 0;
    if (S != SOCKET_INVALID) socket_close(S);
    socket_destroy();
}
```

Step 3 增加多线程日志互斥锁

```
// 当子线程需要使用日志，先加锁，等完成后，再解锁。
void log_mutex_func(int lock, void *ctx) {
    Mutex *mutex = (Mutex *)ctx;
    if (lock) mutex_lock(mutex);
    else mutex_unlock(mutex);
}
```

Step 4 增加信息保存至 CSV 表格与信息统计两个回调函数

```
void log_callback_csv(LogEvent *event);
void log_callback_statistics(LogEvent *event);
```

Step 5 启动 HTTP Web Server

```
// SOCK_STREAM 代表 socket 使用 TCP 协议。
socket_create(AF_INET, SOCK_STREAM, 0);

// 设置端口 SO_REUSEADDR 复用。
socket_setopt(S, SOL_SOCKET, SO_REUSEADDR).

// 绑定端口后监听端口。
socket_bind(S, &server, sizeof(server));
socket_listen(S, 16);
```

Step 6 等待接收来自客户端的请求并创建子线程处理

```
// socket 阻塞接收来自客户端的请求。
Socket c = socket_accept(S, &client, &client_length);

// 为客户端请求创建子线程，交给子线程处理。
Thread thread;
thread_create(&thread, route_coroutine, (void *) (uintptr_t)c)

// 线程分离，处理客户端请求具备独立的生命周期，非阻塞计算，待处理完成后回收资源。
thread_detach(&thread);
```

Step 7 路由协程处理客户端子线程

```
// 用户上下文指针强制转换。
Socket c = (Socket)(uintptr_t)args;

// 接收客户端的请求内容。
int n = socket_recv(c, buffer, sizeof(buffer) - 1, 0);

// 开启互斥锁。
mutex_lock(&mutex);
// 解析 GET 请求的路径，如果文件字节数据已存在 HashMap 字典中，直接响应客户端，否则尝试读取该文件并加入到 HashMap 字典。
if (strcmp(url, "/") == 0) header_length = http_header(header, sizeof(header), strlen(body), 200);
else if (strcmp(url, "/index.html", 12) == 0) header_length = http_header(header, sizeof(header), strlen(body), 200);
else header_length = http_header(header, sizeof(header), strlen(body), 404);
// 解除互斥锁。
mutex_unlock(&mutex);
```

Step 8 如果包含 Range 字段则解析范围

```
sscanf(range, "Range: bytes=%d-%d", &range_start, &range_end);
body = os_readfile(..., range_start, range_end);
header_length = http_header(header, sizeof(header), strlen(body), 206);
```

Step 9 构建 HTTP 响应体并发送

```
// 构建成功的 HTTP 头。
header_length = snprintf(
    header, header_size,
    "HTTP/1.1 200 OK\r\n"
    "Content-Type: text/html\r\n"
    "Content-Length: %lu\r\n"
    "Connection: close\r\n"
    "\r\n", content_length
);

// 构建不存在的 HTTP 头。
header_length = snprintf(
    header, header_size,
    "HTTP/1.1 404 Not Found\r\n"
    "Content-Type: text/html\r\n"
    "Content-Length: %lu\r\n"
    "Connection: close\r\n"
    "\r\n", content_length
);

// 构建内部服务器的 HTTP 头。
header_length = snprintf(
    header, 256,
    "HTTP/1.1 500 Internal Server Error\r\n"
    "Content-Type: text/html\r\n"
    "Content-Length: %lu\r\n"
    "Connection: close\r\n"
    "\r\n", content_length
);

socket_send(c, header, header_length, 0);
socket_send(c, body, strlen(body), 0);
socket_close(c);
```

2.4.2 编译执行

在 main.c 同级目录下，我们编写好跨平台的 Makefile 文件，使用 GNU GCC、MinGW GCC、Apple Clang 编译器，在终端执行指令 `make` 进行编译。

```
illusionna@illusionna:~/Downloads/HTTP
total 32
-rw-rw-r--@ 1 illusionna  staff   670B  12  5  15:06 Makefile
drwxrwxr-x@ 7 illusionna  staff   224B  12  5  15:06 UI
-rw-rw-r--@ 1 illusionna  staff   9.8K  12  5  15:06 main.c
drwxrwxr-x@ 12 illusionna  staff   384B  12  5  15:06 utils

illusionna@illusionna:~/Downloads/HTTP v15.0.0 23:06 > make
gcc -c main.c -o main.o -O2 -flto
gcc -c utils/hashmap.c -o utils/hashmap.o -O2 -flto
gcc -c utils/log.c -o utils/log.o -O2 -flto
gcc -c utils/os.c -o utils/os.o -O2 -flto
gcc -c utils/socket.c -o utils/socket.o -O2 -flto
gcc -c utils/thread.c -o utils/thread.o -O2 -flto
gcc ./main.o ./utils/hashmap.o ./utils/log.o ./utils/os.o ./utils/socket.o ./utils/thread.o -o httpserver -O2 -flto -pthread

illusionna@illusionna:~/Downloads/HTTP v15.0.0 23:06 > ll
total 176
-rw-rw-r--@ 1 illusionna  staff   670B  12  5  15:06 Makefile
drwxrwxr-x@ 7 illusionna  staff   224B  12  5  15:06 UI
-rwxr-xr-x@ 1 illusionna  staff   52K  12  5  23:06 httpserver
-rw-rw-r--@ 1 illusionna  staff   9.8K  12  5  15:06 main.c
-rw-rw-r--@ 1 illusionna  staff   17K  12  5  23:06 main.o
drwxrwxr-x@ 17 illusionna  staff   544B  12  5  23:06 utils

illusionna@illusionna:~/Downloads/HTTP v15.0.0 23:06 > _
```

图 1: 编译 HTTP Web 服务器

终端执行二进制 httpserver(.exe) 启动 Web 服务，默认为 http://localhost:8080 网址。

2.4.3 网络测试

本地回环测试 客户端的目标 IPv4 为 localhost，默认端口为 8080，通过向本地服务器发送 curl 请求，以验证 HTTP 是否正常响应。

局域网内网测试 将客户端目标 IPv4 修改成 ipconfig / ifconfig 获得到的局域网地址，通过局域网访问 Web 服务器是否能响应。

互联网公网测试 使用公网 38.147.173.246 服务器（预计 2026 年失效）的 8080 端口，将程序服务端部署在公网环境，本地客户端手机通过 UCAS 网络 10.207.130.55 发送请求，查看浏览器是否成功渲染响应的 HTML 页面。

请输入IP或网站域名:

IP 地址:	38.147.173.246
IP Long:	647212534

高精归属地定位 (IP数据云)

中国 香港 香港

归属地(纯真数据):	美国 加利福尼亚州 洛杉矶 洛杉矶 Cogent
归属地(ipip):	美国 美国 -
归属地(淘宝数据):	美国
归属地(IP2REGION):	美国 科进
归属地(GeoLite2):	United States -
归属地(DbIp):	United States -

图 2: 位于美国洛杉矶的公网服务器（VPN 代理节点位于中国香港）

测试样例 是否响应 HTML 页面内容、如何处理不存在的内容、响应特定范围的内容、拒绝非法访问内容。

```
curl http://localhost:8080
curl http://localhost:8080/index.html
curl http://localhost:8080/nonexistent.html
curl http://localhost:8080/LICENSE/MIT
curl http://localhost:8080/LICENSE/../../main.py
curl http://localhost:8080/LICENSE/../../log.csv
curl -r 7-12 http://localhost:8080/index.html
curl -r 7-12 http://localhost:8080/index.html -o part.html
curl -r 32-64 http://localhost:8080/LICENSE/GNU-GPL-3.0
```

2.4.4 效果截图

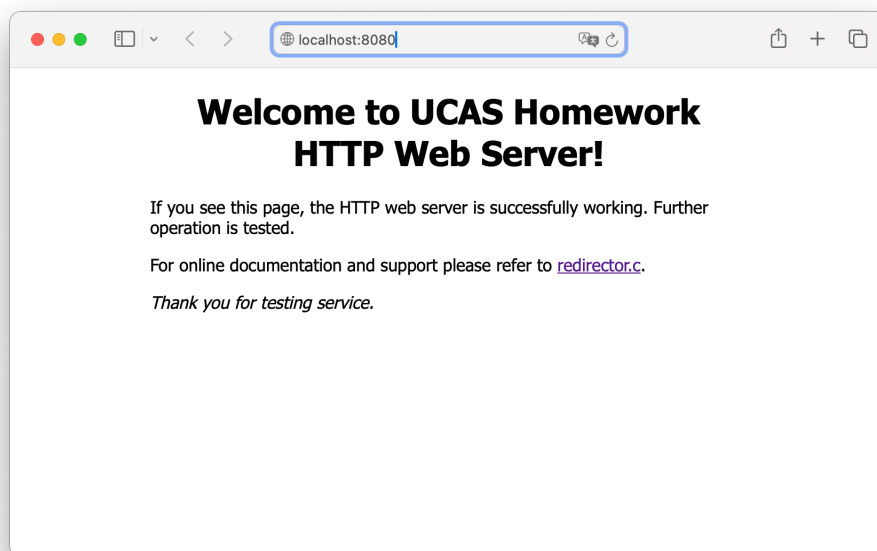


图 3: HTTP UCAS HTML

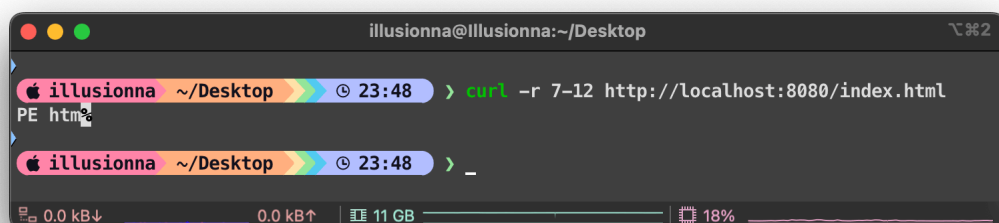


图 4: curl 请求 Range 字段本地回环测试

```
illusionna@illusionna:~/Downloads/HTTP
INFO: Started server process ID [68070]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: C socket service running on http://localhost:8080 (Press CTRL+C to quit)
2025-12-05 23:43:41 TRACE main.c:267: client (127.0.0.1:65289)
2025-12-05 23:43:41 TRACE main.c:267: client (127.0.0.1:65291)
^C
* Interrupt: Disconnect Socket and Free Memory.
bucket_0 --> (null)
bucket_1 --> (null)
bucket_2 --> (null)
bucket_3 --> (null)
bucket_4 --> {./UI/404.html: ...} --> {./UI/default.html: ...} --> (null)
bucket_5 --> (null)
bucket_6 --> (null)
bucket_7 --> (null)
HashMap Dictionary Information: buckets = 8, count = 2, load_factor = 25.00%
INFO: Shutting down.
INFO: Waiting for application shutdown.
INFO: Application shutdown complete.
INFO: Finished server process ID [68070]
```

图 5: HTTP Web 服务器终端信息预览

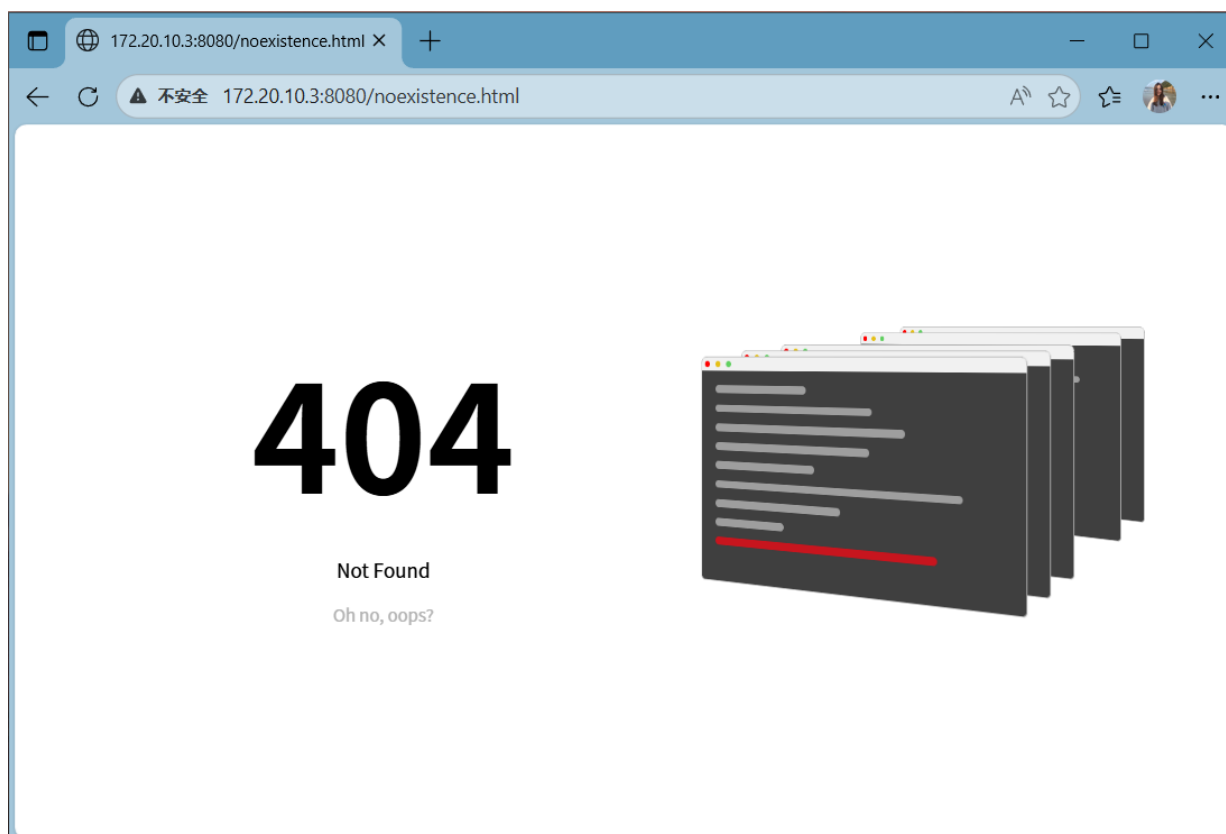


图 6: 局域网下 Windows 10 请求 macOS Sonoma 服务器不存在的文件

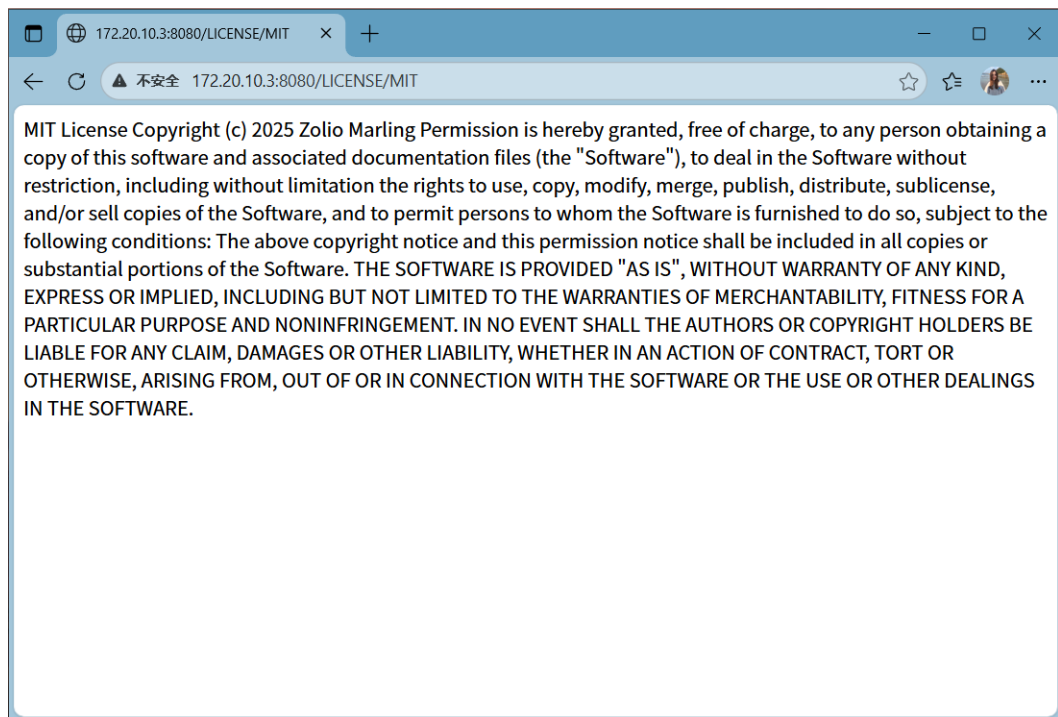


图 7: 局域网下 Windows 10 请求 macOS Sonoma 服务器子目录下的文件

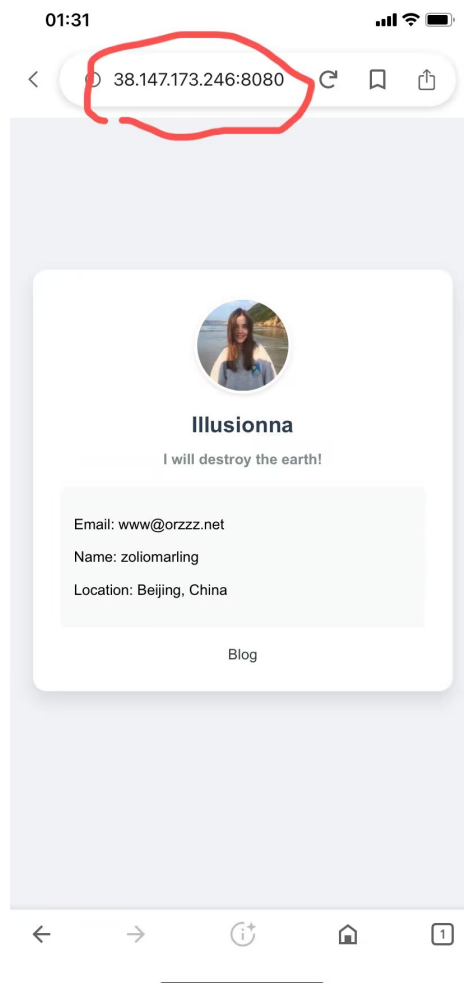


图 8: 公网环境下通过手机访问

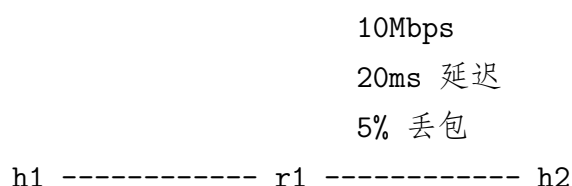
3 基于 UDP 的可靠数据传输

3.1 项目描述

在 UDP 之上实现一个你自己的简单可靠传输协议：

- 序列号 SEQ 来保证顺序；
- 确认 ACK 来保证送达；
- 超时重传机制；
- 简单的流量控制（滑动窗口机制、接收窗口调整）。

3.2 拓扑结构



3.3 实验流程

- 在 h1 的 xterm 中，运行你实现的 RDT 程序的服务器模式，用于接收数据；
- 在 h2 的 xterm 中，运行你实现的 RDT 程序的客户端模式，用于向服务器发送数据（譬如一个 10MB 左右大小的文件）；
- 观察你的 RDT 协议是否能正确处理丢包，使用 md5sum 验证文件是否完整有序地送达；
- 计算平均传输速率与可用带宽（10Mbps）有多大差距。

3.4 项目

3.4.1 代码设计

本实验旨在基于无连接、不可靠的 UDP 协议，在应用层实验一个具备可靠传输特性的协议（RDT），通过该实验，我们深入理解了传输控制协议中的核心机制，包括序列号（SEQ）管理、确认应答（ACK）、超时重传（RETRY）以及流量控制（滑动窗口）机制。

实验设定的网络拓扑环境具有一定的挑战性：链路带宽限制为 10Mbps，存在 20ms 的传输延迟，且有 5% 的丢包率。这种环境模拟了真实网络中可能出现的不稳定情况，要求我们的协议必须具备较强的稳定性。

为了克服 UDP 的不可靠性，我们设计了自定义报文结构：包含长度、序列号和 1KB 数据载荷的 PacketUDP，以及包含确认号和窗口大小的确认报文 PacketACK。

在可靠性保障方面，发送方为每个数据包分配递增的序列号，接收方通过累积确认（Cumulative ACK）告知发送方期望的下一个序列号。通信开始时，客户端发送包含 PING 的特殊序列号包，服务器回复初始 ACK 以完成连接建立。一旦发生超时（默认为 200ms），发送方采用 Go-Back-N（回退 N 步）策略，重传当前窗口内所有未被确认的数据包。

在流量控制方面，我们实现了固定滑动窗口机制。发送方维护 WINDOW_SIZE，其大小为 10 的窗口，只有当接收方回复 ACK 确认已收到数据后，窗口才会向前滑动。这种机制限制了发送方同一时刻在网络中传输的数据包数量，在一定程度上起到了发送速率控制的作用。

服务器端代码设计主要过程如下：

Step 1 初始化全局参数

```
#define PORT 1314
#define MAX_RETRY 5
#define TIMEOUT_MS 200
#define WINDOW_SIZE 10
#define BUFFER_SIZE 1024

static volatile sig_atomic_t RUNNING = 1;
static Socket S = SOCKET_INVALID;
static FILE *F = NULL;
static HashMap *dict = NULL;
```

Step 2 启动 Server 服务器端

```
// SOCK_DGRAM 代表使用 UDP 协议。
S = socket_create(AF_INET, SOCK_DGRAM, 0);
```

Step 3 服务器接收来自客户端的 UDP 数据包

```
int n = socket_recvfrom(S, &udpong, sizeof(udpong), 0, &client, &client_length);
```

Step 4 如果 UDP 包序列号为 0xFFFFFFFF，即 unsigned int 最大值

```
// 发送任意数据，表明服务器能够收到客户端信息，用于检测握手协议。
socket_sendto(...);
```

Step 5 如果 UDP 包长度为 0，则接收到最后一个数据包

```
// 检测文件 MD5 哈希值，如果哈希值已经存在于字典，则向客户端发送“文件已存在的信息”。
for (int i = 0; i < 5; i++) {
    socket_sendto(S, &information, sizeof(information), 0, &client, client_length);
    os_sleep(0.04);
}
// 每隔 40ms 发送一次，连续发送 5 次，但凡有一次被客户端成功接收，则表明文件存在。
// 传输 5 次，是为了将成功概率提升  $\Pr(\text{success}) = 1 - \prod_{i=1}^5 \Pr(\text{failure})$ 。

// 如果文件不存在，则给客户端发送服务器记录的信息。
snprintf(
    information, sizeof(information),
    "Server information:\n - md5 hash: %s\n - average speed: %.2f Mbps\n - total UDP packets received: %u\n - duplicate packets: %u\n\n"
    - out of order packets: %u\n - total size received: %s\n - transmission time: %.5f s",
    hash, speed_mbps, received_packets, duplicate_packets, out_of_order_packets, filesize, elapsed_time
);
log_info(information);

// 重置所有参数记录，并跳过本次循环。
received_packets = 0;
expected_sequence = 0;
cumulative_bytes = 0;
duplicate_packets = 0;
out_of_order_packets = 0;
skip = 0;
continue;
```

Step 6 服务器已接收的 UDP 数据包计数

```
received_packets++;
```

Step 7 服务器接收序列号等于期待序列号

```
// 如果接收到的序列号不等于期待序列号，不做任何处理。  
// 如果相等，如果是 0 号序列号，则表明接收到文件名，如果是 1 号序列号，则表明接收到文件 MD5 哈希值，否则表明接收到文件内容，则写入磁盘。
```

```
if (udpong.sequence == 0) {  
    memcpy(filename, udpong.data, udpong.length);  
} else if (udpong.sequence == 1) {  
    hashmap_get(dict, udpong.data);  
    hashmap_put(dict, udpong.data, filename);  
} else {  
    fwrite(udpong.data, 1, udpong.length, F);  
}
```

Step 8 服务器给客户端发送自己期待下一次接收到的序列号

```
udping.acknowledge = socket_htonl(expected_sequence);  
udping.window_size = socket_htonl(WINDOW_SIZE);  
socket_sendto(S, &udping, sizeof(udping), 0, &client, client_length);
```

客户端代码设计主要过程如下：

Step 1 设置超时处理

```
Time duration = {  
    .second = 0,  
    .microsecond = TIMEOUT_MS * 1000  
};  
// SOCK_DGRAM 代码使用 UDP 协议。  
Socket c = socket_create(AF_INET, SOCK_DGRAM, 0);  
socket_setopt(c, SOL_SOCKET, SO_RCVTIMEO, &duration, sizeof(duration));
```

Step 2 客户端与服务器进行握手，检测是否能抵达

```
handshake(c, &server, server_size);
```

Step 3 读取文件并分配给每个 UDP 数据包

```
// 第一个 UDP 包是文件名。  
cache[0].sequence = 0;  
strncpy(cache[0].data, basename, BUFFER_SIZE - 1);  
  
// 第二个 UDP 包是文件 MD5 哈希。  
cache[1].sequence = 1;  
strncpy(cache[1].data, hash, BUFFER_SIZE - 1);  
  
cache[cumulative_packets].sequence = cumulative_packets;  
unsigned int length = fread(cache[cumulative_packets].data, 1, BUFFER_SIZE, f);
```

Step 4 客户端向服务器端发送 UDP 数据包

```
// 窗口偏移量未超过总共包数量。  
while (window_offset < cumulative_packets) {  
    // 一次性将滑动窗口内的所有 UDP 包全部发送给服务器。  
    while (next_sequence < window_offset + window_size && next_sequence < cumulative_packets) {  
        PacketUDP udping = cache[next_sequence];  
        udping.sequence = socket_htonl(udping.sequence);  
        udping.length = socket_htonl(udping.length);  
        socket_sendto(c, &udping, sizeof(udping), 0, &server, server_size);  
        next_sequence++;  
    }  
}
```

Step 5 客户端接收来自服务器端响应

```
PacketACK udpong;
int n = socket_recvfrom(c, &udpong, sizeof(udpong), 0, NULL, NULL);

// 如果有响应内容, 则滑动窗口偏移量移动到服务器期待收到的序列号位置.
if (n > 0) {
    window_offset = acknowledge;
} else {
    // 如果没有响应内容, 判断是否超时.
    if ((now_time - last_time) * 1000 > TIMEOUT_MS) {
        retry++;
        // 如果超过最大超时次数, 则退出.
        if (retry > MAX_RETRY) return 1;
        // 下一个序列号回退到滑动窗口偏移量位置.
        next_sequence = window_offset;
        cumulative_retransmission++;
    }
}
```

Step 6 发送结束标志的 UDP 数据包

```
// 给最后一个 UDP 数据包内容全部置为 0
memset(&request, 0, sizeof(request));
request.sequence = socket_htonl(cumulative_packets);
retry = 0;

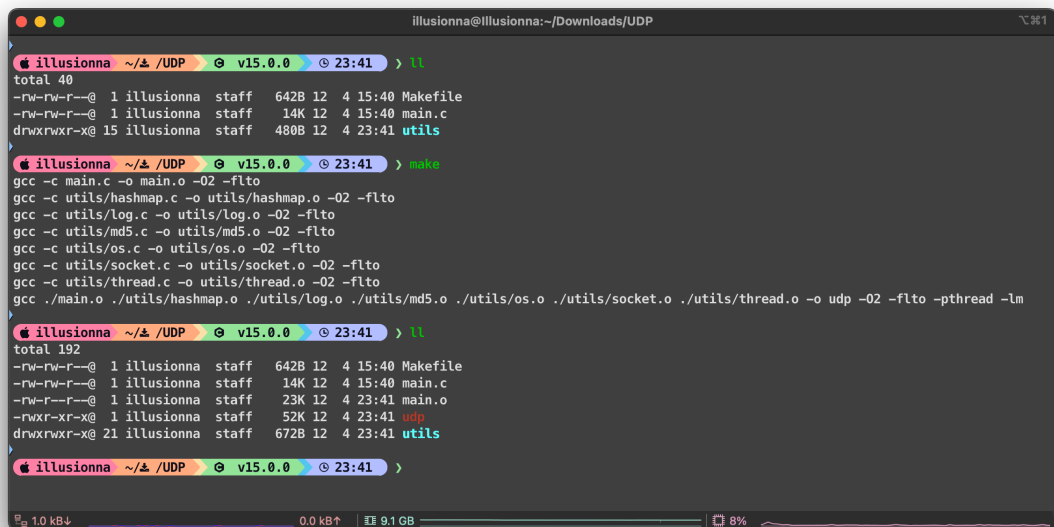
// 此处我的设计也是为了提高成功发送概率, 最多重传 5 次,  $\Pr(\text{success}) = 1 - \prod_{i=1}^5 \Pr(\text{failure})$ .

while (retry < MAX_RETRY) {
    socket_sendto(c, &request, sizeof(request), 0, &server, server_size);
    char response[BUFFER_SIZE];
    int n = socket_recvfrom(c, &response, sizeof(response), 0, NULL, NULL);
    // 由于服务器连续发送 5 次终止数据包, 因此这里只需要接收到其中一个包, 即可退出循环.
    if (n > 0) break;
    retry++;
}

// 注意, 客户端最终会计算本地的文件 MD5 哈希值, 并且与服务器发送来的 MD5 哈希值进行对比.
```

3.4.2 编译执行

在 main.c 同级目录下, 我们编写好跨平台的 Makefile 文件, 使用 GNU GCC、MinGW GCC、Apple Clang 编译器, 在终端执行指令 `make` 进行编译。



```
illusionna@illusionna:~/Downloads/UDP
❯ ll
total 40
-rw-rw-r--@ 1 illusionna staff 642B 12 4 15:40 Makefile
-rw-rw-r--@ 1 illusionna staff 14K 12 4 15:40 main.c
drwxrwxr-x@ 15 illusionna staff 480B 12 4 23:41 utils

illusionna@illusionna:~/Downloads/UDP
❯ make
gcc -c main.c -o main.o -O2 -flto
gcc -c utils/hashmap.c -o utils/hashmap.o -O2 -flto
gcc -c utils/log.c -o utils/log.o -O2 -flto
gcc -c utils/md5.c -o utils/md5.o -O2 -flto
gcc -c utils/os.c -o utils/os.o -O2 -flto
gcc -c utils/socket.c -o utils/socket.o -O2 -flto
gcc -c utils/thread.c -o utils/thread.o -O2 -flto
gcc -c ./main.o ./utils/hashmap.o ./utils/log.o ./utils/md5.o ./utils/os.o ./utils/socket.o ./utils/thread.o -o udp -O2 -flto -pthread -lm

illusionna@illusionna:~/Downloads/UDP
❯ ll
total 192
-rw-rw-r--@ 1 illusionna staff 642B 12 4 15:40 Makefile
-rw-rw-r--@ 1 illusionna staff 14K 12 4 15:40 main.c
-rw-rw-r--@ 1 illusionna staff 23K 12 4 23:41 main.o
-rw-rw-r--@ 1 illusionna staff 52K 12 4 23:41 udp
drwxrwxr-x@ 21 illusionna staff 672B 12 4 23:41 utils

illusionna@illusionna:~/Downloads/UDP
❯
```

图 9: 编译 UDP 可靠传输项目

二进制程序 `udp.exe` 有两种模式，服务器模式和客户端模式。使用 `./udp.exe run` 运行服务器，使用 `./udp.exe /Users/illusionna/Desktop/main.pdf` 运行客户端给服务器发送文件。

程序执行的整体流程为，服务器启动 socket 并绑定端口，进入等待接收模式，然后客户端读取并传输文件，并且计算文件的 MD5 哈希值，客户端将文件名封装为 0 号序列 UDP 数据包，将 MD5 封装为 1 号序列数据包，优先发送给服务器，以便服务器建立文件映射。而后，客户端将文件内容切分为 1024 字节数据块，从 2 号序列开始依次按照滑动窗口尺寸发送。传输过程中，系统会记录丢包、乱序、重传的统计信息，传输结束后，通过 MD5 校验算法对比源文件和接收文件的哈希值，验证数据的完整性。服务器将接收端的统计信息（含计算的 MD5、平均速率等）回传给客户端。

3.4.3 网络测试

本地回环测试 客户端的目标 IPv4 为 localhost，默认端口为 1314，通过向本地服务器发送文件，以验证在不丢包的情况下是否能传输完整文件。

局域网内网测试 将客户端目标 IPv4 修改成 ipconfig / ifconfig 获得到的局域网地址，通过局域网访问验证传输结果，这样我们就在两台机器间完成小距离的真实网络传输测试。

互联网公网测试 使用公网 38.147.173.246 服务器（预计 2026 年失效）的 1314 端口，将程序服务端部署在公网环境，本地客户端通过 UCAS 网络 10.207.130.55 发送文件，这样我们就在北京与洛杉矶的两台机器间完成长距离的真实网络传输测试。

3.4.4 效果截图

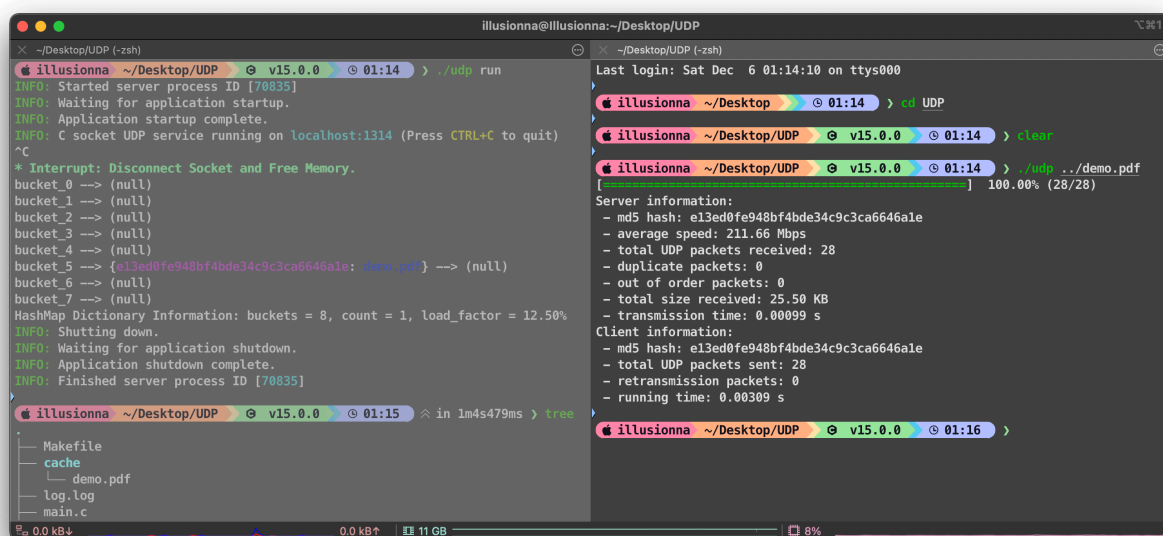


图 10: 服务器 cache 文件夹新增 demo.pdf 文件

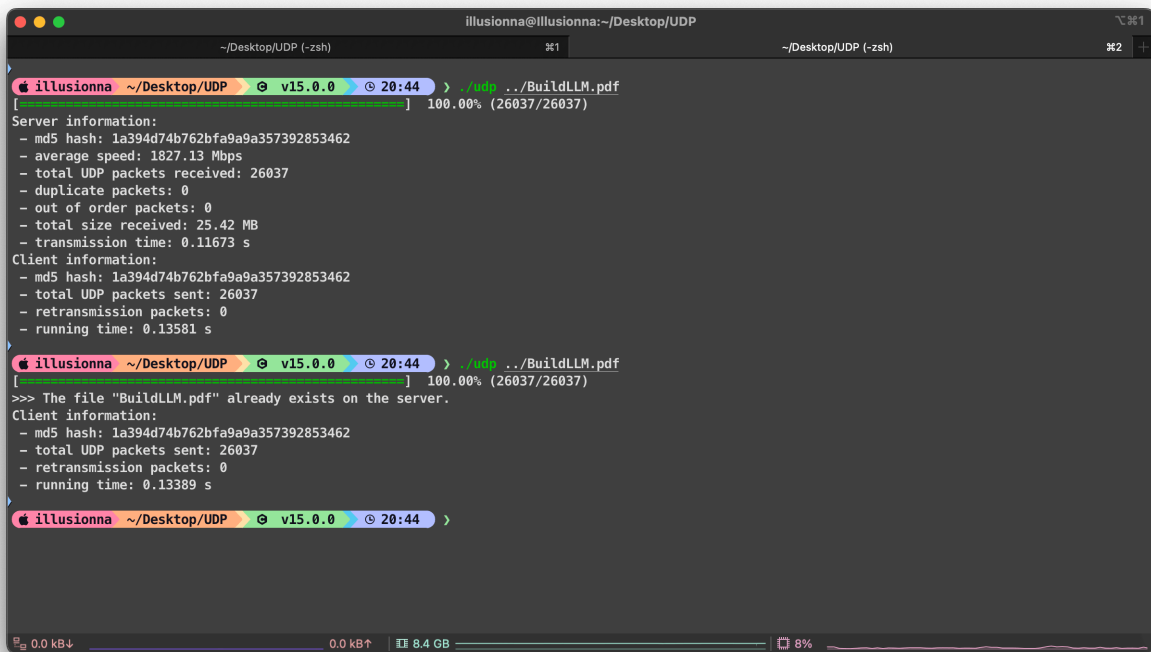


图 11: 重复文件不写入磁盘

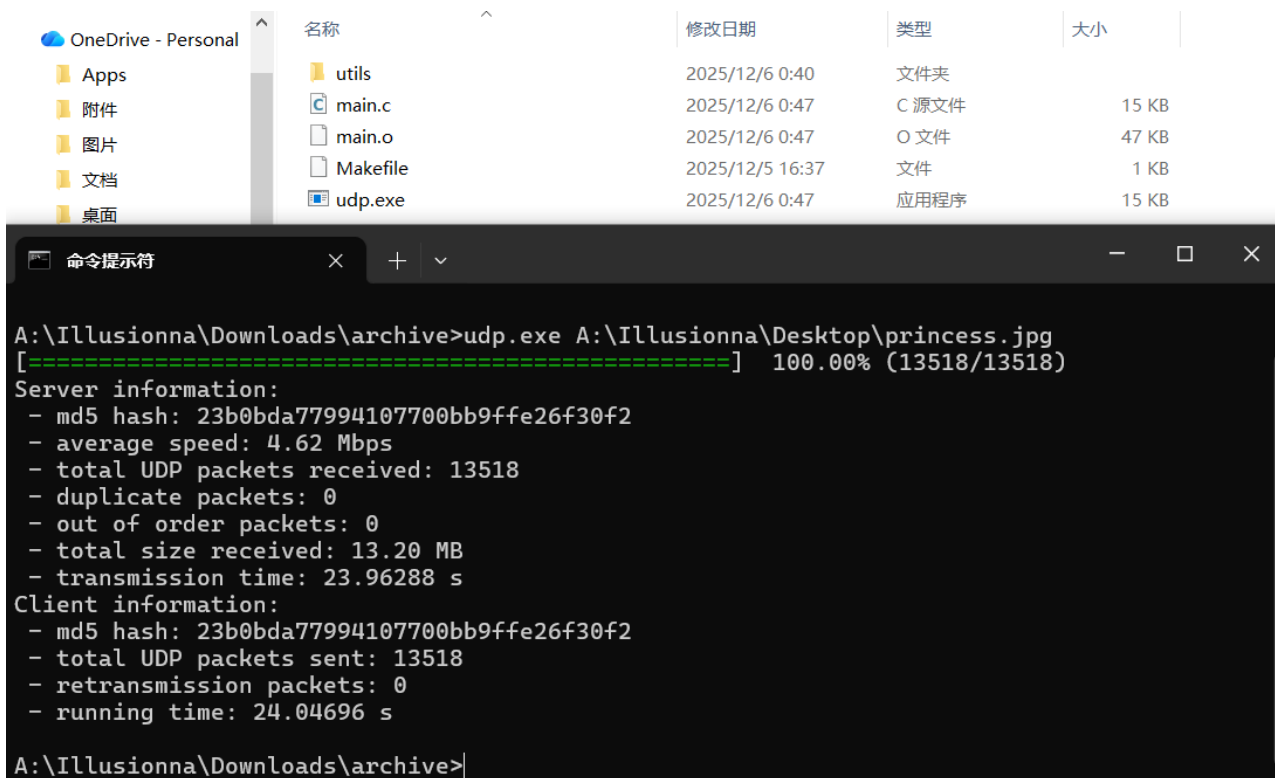


图 12: Windows 10 向 macOS Sonoma 通过 UDP 发送图片（客户端）

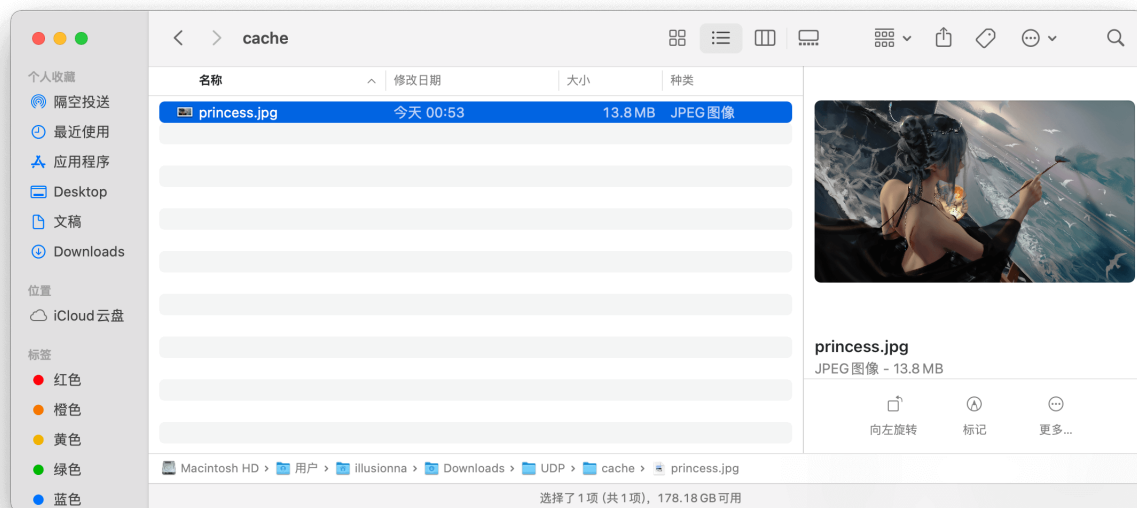


图 13: macOS Sonoma 通过 UDP 接收来自 Windows 10 的图片（服务器）

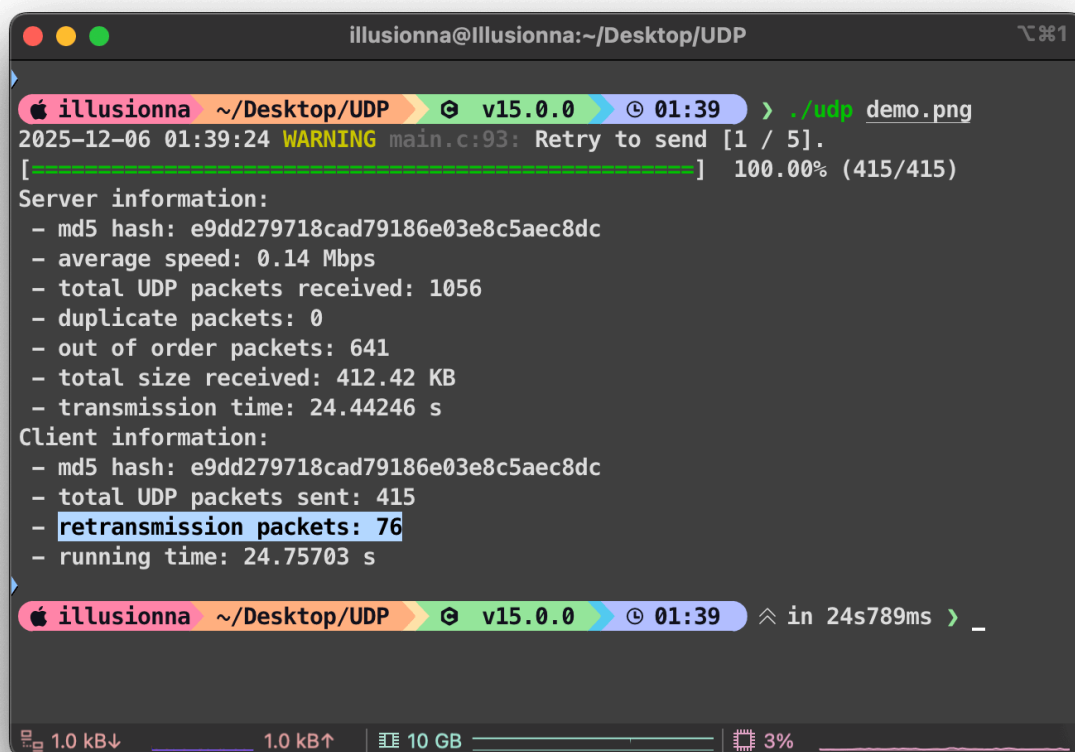


图 14: 注意到公网传输发送丢包重传现象

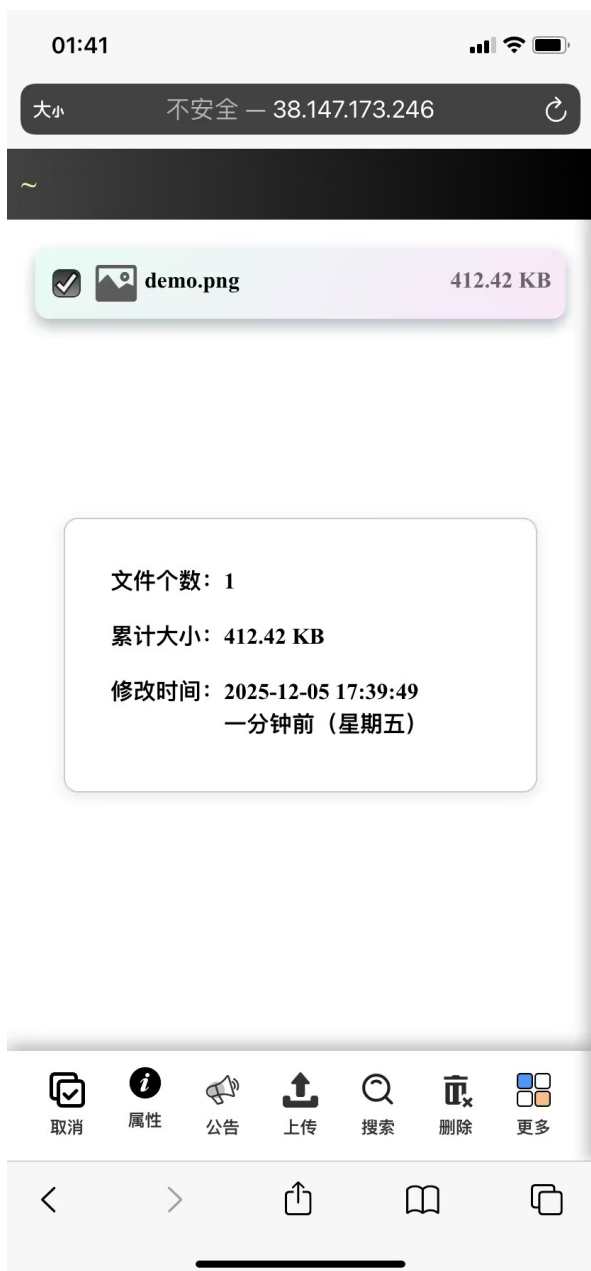


图 15: 公网服务器收到 UCAS 传来的图片

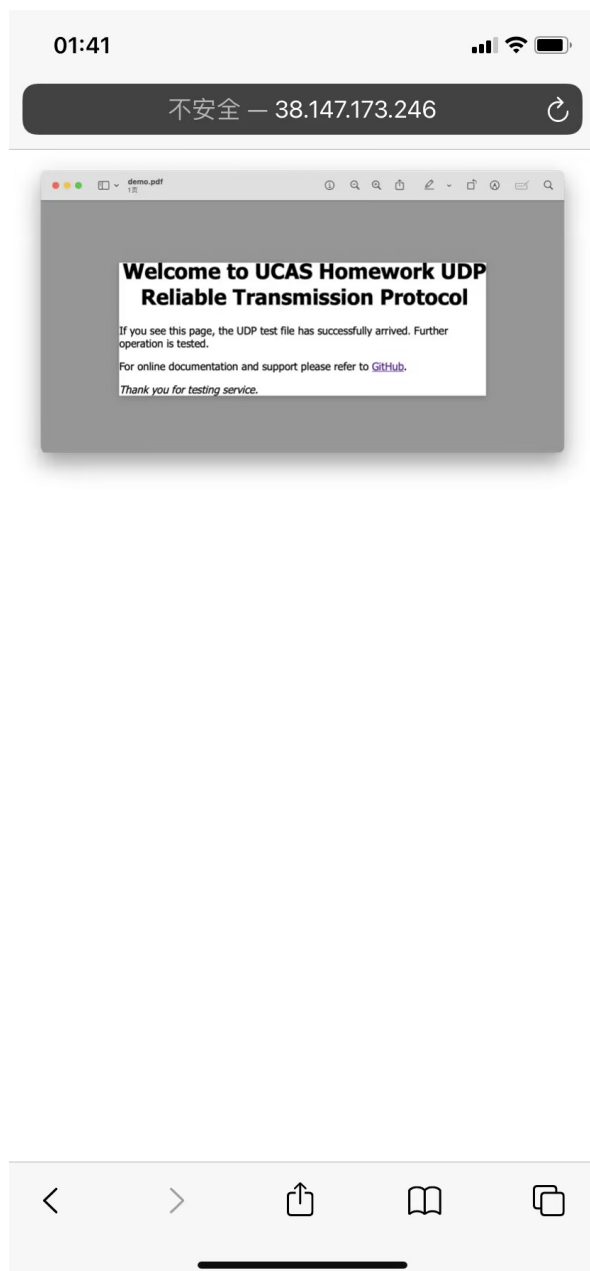


图 16: 查看图片内容

4 实验总结

在设计 HTTP 网络服务器中，我们构建一个功能完备的简易 Web 服务器，实验结果表明，服务器能够解析各种 HTTP GET 请求，并严格按照 HTTP/1.1 协议标准进行响应，不仅验证了 TCP socket 编程的正确性，也加深了我们对 HTTP 协议状态码、头部字段及其交互流程的理解。

在设计 UDP 可靠传输协议中，我们模拟了本地回环、局域内网、互联公网三种环境下的测试，模拟真实丢包情况，我们的协议是否能成功完成文件传输，并且通过 MD5 散列值检验重传机制能有效修复所有丢失的数据包，尽管存在丢包和延迟，但退回 N 步的滑动窗口机制保证了较高的链路利用率，终端显示的平均传输速率，可与实际带宽对比差距，详细的重复包和乱序包统计，直观反映网络状况和协议的对应策略。